

torch.meshgrid#

<https://pytorch.org/docs/stable/generated/torch.meshgrid.html#torch.meshgrid>

Description:

Creates grids of coordinates specified by the 1D inputs in attr:tensors. This is helpful when you want to visualize data over some range of inputs. See below for a plotting example. Given N 1D tensors $T_0 \dots T_{N-1}$ as inputs with corresponding sizes $S_0 \dots S_{N-1}$, this creates N -dimensional tensors $G_0 \dots G_{N-1}$, each with shape $(S_0, \dots, S_{N-1})$ where the output G_i is constructed by expanding T_i to the result shape. 0D inputs are treated equivalently to 1D inputs of a single element.

Function Signature

```
torch.meshgrid(*tensors, indexing=None)[source]#
```

Parameters

Returns

Return type

Returns:

seq (sequence of Tensors)

Code Examples

```
>>> x = torch.tensor([1, 2, 3])
>>> y = torch.tensor([4, 5, 6])
```

Observe the element-wise pairings across the grid, (1, 4), (1, 5), ..., (3, 6). This is the same thing as the cartesian product.

```
>>> grid_x, grid_y = torch.meshgrid(x, y, indexing='ij')
>>> grid_x
tensor([[1, 1, 1],
        [2, 2, 2],
        [3, 3, 3]])
>>> grid_y
tensor([[4, 5, 6],
        [4, 5, 6],
        [4, 5, 6]])
```

This correspondence can be seen when these grids are stacked properly.

```
>>> torch.equal(torch.cat(tuple(torch.dstack([grid_x,
grid_y]))),
...             torch.cartesian_prod(x, y))
True
```

`torch.meshgrid` is commonly used to produce a grid for plotting.

```
>>> import matplotlib.pyplot as plt
>>> xs = torch.linspace(-5, 5, steps=100)
>>> ys = torch.linspace(-5, 5, steps=100)
>>> x, y = torch.meshgrid(xs, ys, indexing='xy')
>>> z = torch.sin(torch.sqrt(x * x + y * y))
>>> ax = plt.axes(projection='3d')
>>> ax.plot_surface(x.numpy(), y.numpy(), z.numpy())
>>> plt.show()
```

Notes & Warnings

NOTE

Note 0D inputs are treated equivalently to 1D inputs of a single element.

⚠ WARNING

Warning `torch.meshgrid(*tensors)` currently has the same behavior as calling `numpy.meshgrid(*arrays, indexing='ij')`. In the future `torch.meshgrid` will transition to `indexing='xy'` as the default. [pytorch/pytorch#50276](https://pytorch.org/docs/stable/torchvision/transforms_pil.html) tracks this issue with the goal of migrating to NumPy's behavior.

NOTE

See also `torch.cartesian_prod()` has the same effect but it collects the data in a tensor of vectors.

Detailed Documentation

Documentation

Creates grids of coordinates specified by the 1D inputs in `attr:tensors`.

This is helpful when you want to visualize data over some range of inputs. See below for a plotting example.

Given N 1D tensors $T_0 \dots T_{N-1}$ as inputs with corresponding sizes $S_0 \dots S_{N-1}$, this creates N -dimensional tensors $G_0 \dots G_{N-1}$, each with shape

$(S_0, \dots, S_{N-1})(S_0, \dots, S_{N-1})$ where the output $G_i G_i$ is constructed by expanding $T_i T_i$ to the result shape.

0D inputs are treated equivalently to 1D inputs of a single element.

`torch.meshgrid(*tensors)` currently has the same behavior as calling `numpy.meshgrid(*arrays, indexing='ij')`.

In the future `torch.meshgrid` will transition to `indexing='xy'` as the default.

[pytorch/pytorch#50276](#) tracks this issue with the goal of migrating to NumPy's behavior.

`torch.cartesian_prod()` has the same effect but it collects the data in a tensor of vectors.

`tensors` (list of Tensor) – list of scalars or 1 dimensional tensors. Scalars will be treated as tensors of size $(1,)(1,)(1,)$ automatically

`indexing` (Optional[str]) –

(str, optional): the indexing mode, either “xy” or “ij”, defaults to “ij”. See warning for future changes.

If “xy” is selected, the first dimension corresponds to the cardinality of the second input and the second dimension corresponds to the cardinality of the first input.

If “ij” is selected, the dimensions are in the same order as the cardinality of the inputs.

If the input has N tensors of size $S_0 \dots S_{N-1}$, then the output will also have N tensors, where each tensor is of shape $(S_0, \dots, S_{N-1})(S_0, \dots, S_{N-1})$.

`seq` (sequence of Tensors)

